Extraire l'identité du patient à grande échelle

NER, BiLSTM, puis un petit LLM fine-tuné

Alexandre Rogojan

Lifen · stage Machine Learning Engineer · partie 2

Le problème, et trois façons de le résoudre

Pour chaque document reçu, extraire quatre champs du patient : **prénom**, **nom**, **nom de naissance** et **date de naissance** [1]. Un champ absent du document donne une réponse vide.

Volume

Des millions de lettres, comptes rendus et notes cliniques par mois [1].

Temps réel

Extraction pendant l'ingestion, l'utilisateur attend : la latence est une contrainte dure [1].

Fiabilité

Une erreur range le document chez un autre patient. Hébergement HDS, sans IA externe [2].

		Principe	Où
1	**NER**	un encodeur étiquette chaque mot	AP-HP [3]
2	**BiLSTM**	un réseau récurrent étiquette chaque mot	Lifen, avant [1]
3	**Petit LLM**	un décodeur écrit les quatre champs	Lifen, aujourd'hui [1]

1.1 NER : méthode, entrée et sortie

Méthode. La reconnaissance d'entités nommées donne une étiquette à chaque mot, avec le schéma BIO [4] : B = début d'une entité, I = suite, O = hors entité.

J'ai	revu	Monsieur	**Jean**	**Pierre**	**DUPONT**
O	O	O	B-PRÉNOM	I-PRÉNOM	B-NOM

Signé : Dr	**Irène**	**ADLER**
O O	B-PRÉNOM	B-NOM

Entrée [3]

- Le texte extrait du PDF, découpé en mots (EDS-NLP), puis en sous-mots.
- Documents longs : fenêtres d'au plus 384 mots, coupées en début de phrase.

Fonctionnement et sortie [3]

- Un encodeur de type CamemBERT [5] lit la fenêtre dans les deux sens ; des têtes CRF choisissent les étiquettes.
- Sortie : des entités typées (12 types), pour *toutes* les personnes.

$$\hat{y}_i = \arg\max_y p_\theta(y \mid x, i), \qquad \mathcal{L}(\theta) = -\sum_{i=1}^{n} \log p_\theta(y_i^\star \mid x, i)$$

Entraînement : 3 682 documents annotés en deux passes, par 3 annotateurs relus par 3 autres [3].

1.2 NER : résultats mesurés

Jeu de test de l'AP-HP [3], par type d'entité ; noms de toutes les personnes (patients, médecins, tiers). $P = \mathrm{VP}/(\mathrm{VP}+\mathrm{FP})$, $R = \mathrm{VP}/(\mathrm{VP}+\mathrm{FN})$, $F_1 = 2PR/(P+R)$.

Entité	P	R	F_1
Prénom	98,8	98,4	98,6
Nom	98,6	98,3	98,4
Date de naissance	98,0	98,2	98,1
Toutes (12 types)	99,1	98,8	99,0

- **Par document** : toutes les entités trouvées dans 84,4 % des documents.
- **Données** : le rappel plafonne vers 1 500 documents d'entraînement, le score par document progresse jusqu'à 3 000.
- **Courbe ROC** : non publiée ; l'article n'analyse aucun score de confiance.

1.3 NER : forces, limites et cas limites

Forces

- Une passe, rapide, possible sur CPU.
- N'invente pas de nom : il étiquette les mots du texte.
- Rappel de 98,4 % sur les prénoms, contre 36,9 % pour les règles [3].
- Outils mûrs pour le français médical : EDS-NLP, CamemBERT.

Limites

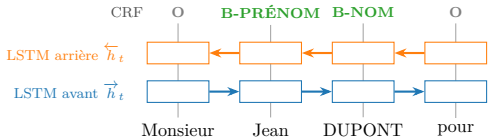
- Annotation mot par mot : 3 682 documents à l'AP-HP.
- Trouve toutes les personnes : il faut encore choisir le patient.
- Des étiquettes, pas des champs : à recoller, à mettre en forme.
- Fenêtre de 384 mots.
- Dépend de l'extraction du texte du PDF.

Cas limites

- Le médecin a les mêmes étiquettes que le patient.
- Noms communs : Pierre, Blanc, Rose.
- Mots collés (« JamesSmith » [3]), noms coupés.
- Nom de naissance : « DUPONT, née MARTIN ».
- Noms composés, initiales : LE GALL, J. DUPONT.

2.1 BiLSTM : méthode, entrée et sortie

Méthode. Toujours une étiquette par mot, avec un réseau récurrent : un LSTM [6] lit les mots un par un et tient une mémoire c_t ; deux LSTM lisent dans les deux sens ; une couche CRF choisit la suite d'étiquettes [7].



- **Entrée** : un vecteur par mot, plongement du mot (100 dimensions) et plongement de ses caractères, lus par un petit BiLSTM (2×25) [7,8].
- Chez Lifen : texte normalisé, appariement de mots, plongements appris de zéro [1].
- **Sortie** : une étiquette par mot ; les noms sur plusieurs mots sont recollés à la main [1].

$$(f_t, i_t, o_t) = \sigma(W[h_{t-1}; x_t] + b)$$
$$\tilde{c}_t = \tanh(W_c[h_{t-1}; x_t] + b_c)$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$
$$h_t = o_t \odot \tanh(c_t), \quad h_t = \left[\overrightarrow{h}_t; \overleftarrow{h}_t\right]$$
$$s(x, y) = \sum_t \left(P_{t,y_t} + A_{y_{t-1},y_t}\right)$$
$$p(y \mid x) = \frac{e^{s(x,y)}}{\sum_{y'} e^{s(x,y')}}$$

f, i, o : portes d'oubli, d'entrée et de sortie.
$P_{t,y}$: score de l'étiquette y pour le mot t ; A : score des transitions (jamais I-NOM après O).
Décodage : $\hat{y} = \arg\max_y s(x, y)$, par Viterbi.

2.2 BiLSTM : résultats mesurés

Système	Données (test)	Niveau	P	R	F_1
BiLSTM-CRF [7]	CoNLL-2003, anglais [9]	entité	n.p.	n.p.	90,9
BiLSTM sans CRF [7]	CoNLL-2003, anglais	entité	n.p.	n.p.	89,2
BiLSTM [8]	i2b2 2014, 1 304 notes	token	98,3	97,4	97,8
CRF classique [8]	i2b2 2014	token	98,6	96,5	97,5
BiLSTM [8]	MIMIC, 1 635 notes	token	99,2	99,3	99,2
BiLSTM de Lifen [1]	documents Lifen, 4 champs	champ (macro)	**96,0**	n.p.	n.p.

- CoNLL-2003 : personnes, lieux, organisations ; en allemand, néerlandais et espagnol, F_1 de 78,8 à 85,8 [7]. La couche CRF ajoute 1,8 point de F_1.
- i2b2 et MIMIC : dé-identification de dossiers médicaux, chaque token est identifiant (nom, date, lieu…) ou non. Le BiLSTM bat le CRF classique en rappel.
- **Courbe ROC** : publiée dans aucun de ces articles ; Lifen ne publie ni le rappel ni le F_1 de son BiLSTM.

n.p. : non publié. Valeurs du tableau 3 de [8] ; son résumé donne $P = 97{,}32$ pour i2b2.

2.3 BiLSTM : forces, limites et cas limites

Forces

- Léger et rapide, sur CPU.
- N'invente pas de nom.
- Contexte dans les deux sens ; les caractères aident pour les mots rares.
- Le CRF évite les suites d'étiquettes impossibles.
- F_1 de 97,8 à 99,2 en dé-identification [8].

Limites

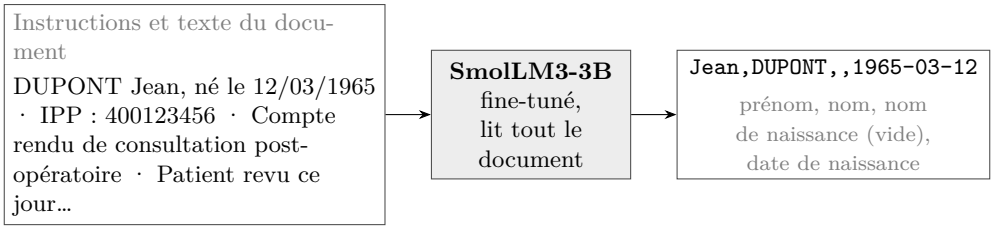
- Chez Lifen : plongements appris de zéro, normalisation et appariement écrits à la main, 96,0 % de précision macro [1].
- Lit mot après mot : le contexte lointain s'efface, le calcul ne se parallélise pas.
- Étiquettes à recoller ; un nouveau champ demande annotations et code.

Cas limites

- « M. Parkinson » est un nom, « maladie de Parkinson » non [8].
- Nom sur plusieurs mots ou plusieurs lignes.
- « DUPONT Jean » ou « Jean DUPONT ».
- Erreurs d'OCR, mots coupés.
- Patient, médecin et proche dans la même lettre.

3.1 Petit LLM : passer au génératif

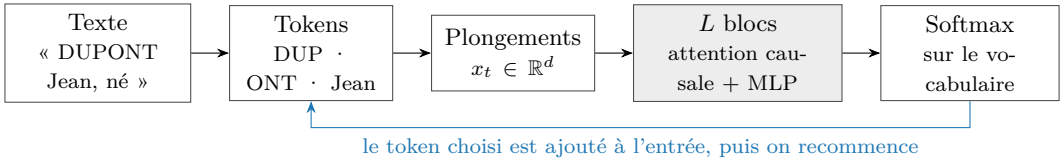
- Le BiLSTM demandait de la normalisation de texte, de l'appariement de mots, des plongements appris de zéro, et un recollage manuel des noms sur plusieurs tokens [1].
- Lifen a écarté les encodeurs de type BERT : un seul modèle qui écrit directement la bonne réponse est plus simple [1].



Ce qui disparaît
Normalisation, appariement, recollage : un seul modèle, une seule sortie.

Ce qui apparaît
Un GPU, la quantification, un serveur d'inférence, des sorties à contrôler.

3.2 Comment fonctionne un LLM : un transformeur décodeur



- **Tokens** : des morceaux de mots, chacun avec son vecteur.
- **Attention causale** [10] : chaque token regarde les tokens précédents, jamais les suivants.
- **Pré-entraînement** : prédire le token suivant, sur 11 200 milliards de tokens pour SmolLM3 [11].

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

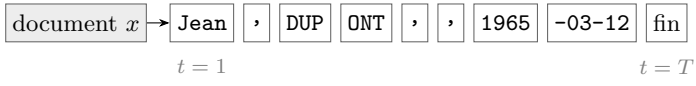
$$M_{ij} = 0 \text{ si } j \le i, \quad -\infty \text{ sinon}$$

$$p_\theta(y_t \mid y_{<t}, x) = \text{softmax}(W_{\text{out}}\, h_t)$$

3.3 Comment il écrit la réponse : génération autorégressive

Le modèle écrit un token à la fois, sachant le document x et ce qu'il a déjà écrit. Avec une température nulle, il prend toujours le plus probable : même entrée, même sortie.

$$p_\theta(y \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}, x)\,, \qquad \hat{y}_t = \arg\max_v \; p_\theta(v \mid \hat{y}_{<t}, x)$$



Découpage en tokens illustratif.

Préremplissage
Instructions et document lus en une passe parallèle. Les clés et valeurs de l'attention sont gardées : c'est le cache KV.

Décodage
Une passe complète du modèle par token écrit, en relisant tous ses poids.

$$\text{latence} \approx t_{\text{préremplissage}} + T \cdot t_{\text{token}}$$

D'où deux leviers : écrire moins de tokens (T, § 3.6) et rendre chaque token moins cher (t_{token}, § 3.7).

3.4 Le modèle choisi : SmolLM3-3B

Architecture [11]	
Type	transformeur décodeur seul
Taille	3 milliards de paramètres
Attention	GQA : cache KV plus petit
Positions	NoPE dans 1 couche sur 4
Contexte	64k tokens, 128k avec YaRN
Langues	6, dont le français
Corpus	11 200 milliards de tokens
Licence	Apache 2.0

Pourquoi ce modèle [1]

- Licence ouverte : hébergé et fine-tuné en interne.
- La précision monte de 98,7 % vers 0,5B paramètres à un plateau de 99,2 % dès 3B ; un modèle de 8B n'apporte rien.
- Un document entier tient dans le contexte.
- GQA : moins de mémoire par requête, donc plus de requêtes en parallèle.

3.5 Obtenir une sortie structurée

Le logiciel attend des champs, pas une phrase. Trois façons de l'obtenir [1] :

Approche	**Principe**	**Limite**
Consignes dans le prompt	décrire le format, donner des exemples	moins fiable par nature [1]
Décodage contraint	interdire les tokens hors format (outlines, guidance, instructor ; schémas JSON des fournisseurs)	ralentit la génération [12]
Fine-tuning	le modèle apprend le format sur des exemples	demande des données annotées

Décodage contraint : à l'étape t, seuls les tokens $\mathcal{A}_t$ permis par la grammaire restent possibles,

$$\tilde{p}(v) = \frac{p_\theta(v \mid y_{<t}, x)\ \mathbf{1}[v \in \mathcal{A}_t]}{\sum_{u \in \mathcal{A}_t} p_\theta(u \mid y_{<t}, x)}$$

Lifen fine-tune le modèle : le format est appris, et la sortie est vérifiée ensuite (§ 3.11).

3.6 Le format de sortie : CSV ou JSON

JSON

```
{"first_name": "John",
 "last_name": "Doe",
 "birth_name": null,
 "birth_date": "1970-01-01"}
```

CSV

```
John,Doe,,1970-01-01
```

Valeurs dans un ordre fixe ; un champ vide laisse une case vide.

- **Même précision** : 99,2 % de précision macro pour les deux formats, avec SmolLM3-3B quantifié en FP8 [1].
- **Moins de tokens** : le CSV n'écrit ni clés, ni guillemets, ni accolades. Moins de passes de décodage, donc une latence plus faible (T plus petit, § 3.3).
- Choix de Lifen : le CSV [1].

Exemples tirés de l'article de Lifen [1].

3.7 Pourquoi quantifier

- **Le problème** : sans quantification, une prédiction prend près d'une seconde [1], alors que l'utilisateur attend pendant l'ingestion.
- **La cause** : au décodage, chaque token oblige le GPU à relire tous les poids. Le temps dépend surtout de la mémoire à lire :

$$t_{\text{token}} \approx \frac{N \cdot b}{B}$$

N : nombre de paramètres
b : octets par paramètre
B : bande passante mémoire

Format	b	Poids	t_{token}
BF16	2	6 Go	≈ 20 ms
FP8	1	3 Go	≈ 10 ms
INT4	0,5	1,5 Go	≈ 5 ms

Ordre de grandeur, mon calcul : $N = 3 \cdot 10^9$, GPU L4 à $B = 300$ Go/s.

- **Mémoire** : des poids plus petits laissent de la place au cache KV et aux lots, sur les 24 Go de la L4.
- **Le risque** : moins de bits, c'est perdre une partie des capacités du modèle [1]. Il faut remesurer la précision après.

3.8 Comment on quantifie : FP8 dynamique, AWQ, GPTQ

Format	Bits	Octets	Max.
BF16	$1+8+7$	2	$3{,}4 \cdot 10^{38}$
FP8 E4M3	$1+4+3$	1	448
INT4	4	0,5	7

Bits : signe + exposant + mantisse ; INT4 : entiers de -8 à 7. Formats FP8 : [13].

Chaque groupe de valeurs x reçoit une échelle s :

$$s = \frac{\max |x|}{q_{\max}}$$

$$x_q = \text{arrondi}\left(\frac{x}{s}\right), \quad \hat{x} = s\, x_q$$

L'erreur $|x - \hat{x}|$ grandit avec s : une seule valeur extrême dégrade tout son groupe.

Méthodes comparées par Lifen, avec llm-compressor [1] :

- **FP8 dynamique (retenu)** : poids en FP8, une échelle par canal, fixée une fois ; activations en FP8, une échelle par token calculée à la volée. Pas de données de calibration.
- **AWQ** [14] : poids seuls, en général sur 4 bits ; activations en 16 bits. Protège les $\approx 1\,\%$ de canaux importants, repérés par la taille des activations, en les mettant à l'échelle avant l'arrondi.
- **GPTQ** [15] : poids seuls ; corrige l'erreur d'arrondi colonne par colonne grâce à une information du second ordre. Décrit par Lifen, pas testé.

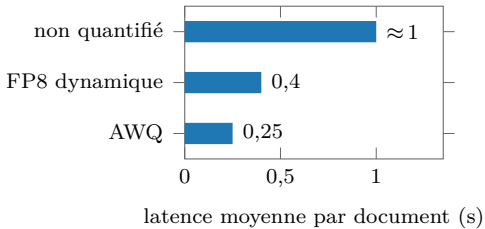
3.9 Servir le modèle : vLLM sur un GPU L4

- **vLLM** [16] : *PagedAttention* range le cache KV en pages, comme la mémoire virtuelle d'un système d'exploitation, donc moins de mémoire perdue ; *batching* continu : une requête rejoint le lot à chaque pas de décodage.
- **Écartés** [1] : TGI, plus maintenu et sans SmolLM3 ; SGLang et TensorRT-LLM sont présentés dans l'article.
- **Matériel** : un GPU NVIDIA L4 par pod ; la latence reste presque constante de 2 à 8 requêtes par seconde [1].
- **Décodage spéculatif** [17] : un petit modèle propose k tokens, le grand les vérifie en une seule passe. Avec un taux d'acceptation α, chaque passe produit en moyenne

$$\mathbb{E}[\text{tokens par passe}] = \frac{1-\alpha^{k+1}}{1-\alpha}$$

  Essayé par Lifen [1] : n-grammes, aucun gain ; brouillon Qwen 0,6B, gain trop faible pour la complexité, SmolLM3 n'ayant pas de petit modèle compagnon. À explorer : EAGLE [18].

3.10 Problèmes : temps d'inférence et précision

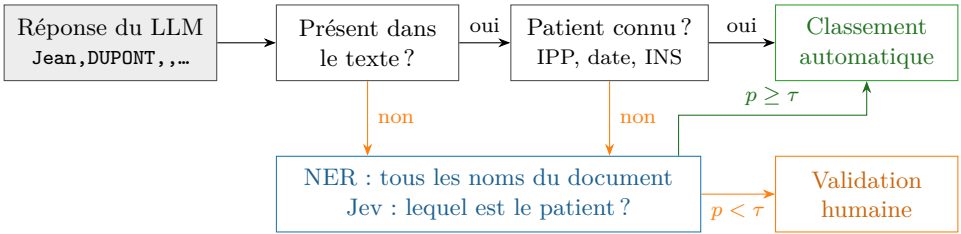


Un pod, GPU L4 [1].

SmolLM3-3B	Précision macro
FP8 dynamique	**99,2 %**
AWQ	un peu moins (n.p.)

- **Temps** : un token après l'autre, pendant que l'utilisateur attend. Un GPU par pod, quand le BiLSTM tournait sur CPU.
- **Précision** : AWQ est le plus rapide, mais perd un peu ; FP8 garde le meilleur score obtenu par Lifen [1].
- **Choix** : FP8 dynamique, pour la précision, une latence sous la seconde et un format pris en charge par les GPU récents [1].
- **Taille** : un modèle de 0,5B irait plus vite, mais tombe à 98,7 %.

3.11 Si le nom en sortie est halluciné ? → Jev



- **LLM peu confiant** : on demande à Jev, avec le document en contexte, « ce candidat est-il le patient ? » (question oui/non).
- **Nom halluciné ou absent** : une NER trouve tous les noms du document, puis Jev choisit le patient parmi eux, avec une probabilité calibrée.
- **Humain** : seulement si Jev est peu confiant ($p < \tau$). Piste récente : les modèles System One datent de 2026 [20].

À valider : τ choisi sur des cas annotés ; Jev est une API externe, à concilier avec l'hébergement HDS [2].

3.12 Comment fonctionne Jev

Entrée : un état et des questions (JSON).

```
état = {
  "texte": "Madame Juliette MARTIN…",
  "candidats": ["Juliette MARTIN",
                "Nicolas JACQUES"]}
question = {"type": "choice",
  "instructions": "Qui est le patient
                   dans `texte` ?",
  "criteria": {
    "Juliette MARTIN": "…",
    "Nicolas JACQUES": "…"}}
```

Sortie : l'option choisie, une probabilité p_c par option, une confiance :

$$\hat{c} = \arg\max_c p_c, \quad \text{confiance} = \frac{p_{\max} - 1/n}{1 - 1/n}$$

- **System One** [20] : pas de texte en sortie, des probabilités sur des questions typées. La réponse est toujours une des options, mais peut être fausse.
- **Quatre types** : `choice` (une option donnée), `noul` (probabilité du oui), `score` (un niveau), `box` (une zone d'image).
- **Calibré** : probabilités entraînées sur les issues réelles (RLCD [21]) ; vrai sur un groupe, pas pour chaque réponse [20].
- **Limites** : options à fournir (255 au plus), API hébergée, architecture non publiée, 32k tokens de contexte [21].

3.13 Le fine-tuning : données et méthode

- **Données** : documents dé-identifiés de types et d'émetteurs variés, annotés par des experts de Lifen et pseudonymisés en interne [1]. Taille : non publiée.
- **Paires** : entrée = instructions et texte du document ; cible = la ligne `prénom,nom,naissance,date`. Objectif du fine-tuning supervisé, token par token :

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log p_\theta(y_t^\star \mid y_{<t}^\star, x)$$

- **LoRA** [19], dans la recherche précédente de Lifen, sur une H100 [12] : on gèle W_0 et on apprend $W = W_0 + BA$, avec $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times d}$ et $r \ll d$. Pour $d = 2048$ et $r = 16$: 65 536 paramètres appris par matrice au lieu de 4,2 millions, soit 1,6 %.
- **Savoir ne rien dire** : un quart d'exemples sans réponse dans cette recherche [12] ; le modèle apprend à laisser un champ vide plutôt qu'à deviner.

Non publiés dans l'article de 2026 : méthode exacte (LoRA ou complète), hyperparamètres, durée d'entraînement.

3.14 Résultats : la précision macro

Définition de Lifen [1] : pour chaque champ k, les prédictions vides sont exclues.

$$P_k = \frac{\text{justes}_k}{\text{non vides}_k}, \qquad P_{\text{macro}} = \frac{1}{4}\sum_k P_k$$

k : prénom, nom, nom de naissance, date de naissance. justes_k : prédictions justes du champ k ; non vides_k : documents où la sortie du champ k n'est pas vide.

- Erreurs, $1 - P_{\text{macro}}$: de 4,0 % à 0,8 %, cinq fois moins ; le titre de l'article annonce 0,6 % (−85 %).
- Tous les modèles fine-tunés sont autour de 99 % [1].
- CSV ou JSON : même précision. FP8 égale le meilleur score de Lifen ; sans quantification : non publié.

Modèle	P_{macro}
BiLSTM (ancien)	96,0 %
LLM ≈ 0,5B	98,7 %
SmolLM3-3B (production)	**99,2 %**
LLM ≈ 8B	99,2 %
SmolLM3-3B, sortie JSON	99,2 %
SmolLM3-3B, AWQ	un peu moins

LLM fine-tunés ; sauf mention, sortie CSV et FP8 dynamique [1].

3.15 Pourquoi pas un grand LLM du marché ?

Comparaison de Lifen sur 500 documents pseudonymisés [1]. Précision par document : la part des documents dont la réponse est entièrement juste.

Modèle	Précision par document	Erreurs	Coût
Opus 4.6, prompt et 5 exemples	86,4 % (432/500)	13,6 %	8,21 $
SmolLM3-3B fine-tuné	**98,5 %**	**1,5 %**	un GPU L4

- Neuf fois moins d'erreurs par document (−89 %) [1].
- Intervalle à 95 % de Wilson pour 432/500 (mon calcul) : $[83,1\,\%; 89,1\,\%]$, avec

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \qquad z = 1{,}96$$

- À l'échelle : 8,21 $ pour 500 documents, environ 16 400 $ par million (mon calcul) ; et les documents partiraient chez un fournisseur externe, ce que Lifen exclut [2].

3.16 Ce que les chiffres publiés ne disent pas

- **Rappel** : non publié,

$$R_k = \frac{\mathrm{VP}_k}{\mathrm{VP}_k + \mathrm{FN}_k}$$

  La précision exclut les réponses vides : un modèle qui s'abstient plus souvent gagne en précision. Il faut aussi le rappel, ou le taux de réponses vides.

- $F_1 = \frac{2PR}{P+R}$ et la précision **par champ** : non publiés.
- **Taille du jeu de test**, intervalles de confiance : non publiés, sauf les 500 documents de la comparaison avec Opus.
- **Courbe ROC** d'un score de confiance : non publiée.

Questions pour l'équipe

- Quel rappel par champ, et quelle part de réponses vides ?
- Qu'est-ce qui envoie un document en validation humaine ?
- Les 0,8 % d'erreurs tiennent-ils pour chaque émetteur et chaque type de document ?

4 Synthèse : NER et petit LLM

	NER (AP-HP)	**Petit LLM (Lifen)**
Mesure publiée	$F_1 = 98{,}6\,\%$ sur les prénoms [3]	précision macro **99,2 %**
Rappel publié	oui : 98,4 % (prénoms)	non
Courbe ROC	non	non
Sortie	étiquettes sur les mots	les quatre champs, mis en forme
Code autour	choisir le patient, mettre en forme	contrôles de la sortie
Matériel	CPU possible	un GPU L4, $\approx 0{,}4\,\mathrm{s}$
Noms inventés	impossible	possibles : contrôles

Tâches et données différentes : ces scores ne sont pas directement comparables entre colonnes.

Références (1/2)

[1] G. Roy, E. Aouad. « Small LLMs, Big Gains: Simpler Patient Extraction with 85% Fewer Errors ». Lifen AI (Substack), 27 juillet 2026.

[2] Lifen. « Intelligence artificielle ». Page lifen.fr, consultée en 2026.

[3] X. Tannier *et al.* « Development and validation of a natural language processing algorithm to pseudonymize documents in the context of a clinical data warehouse ». arXiv:2303.13451, 2023.

[4] L. A. Ramshaw, M. P. Marcus. « Text Chunking using Transformation-Based Learning ». Workshop on Very Large Corpora, 1995.

[5] L. Martin *et al.* « CamemBERT: a Tasty French Language Model ». ACL, 2020.

[6] S. Hochreiter, J. Schmidhuber. « Long Short-Term Memory ». *Neural Computation*, 1997.

[7] G. Lample *et al.* « Neural Architectures for Named Entity Recognition ». NAACL, 2016.

[8] F. Dernoncourt *et al.* « De-identification of patient notes with recurrent neural networks ». *JAMIA*, 2017.

[9] E. F. Tjong Kim Sang, F. De Meulder. « Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition ». CoNLL, 2003.

[10] A. Vaswani *et al.* « Attention Is All You Need ». NeurIPS, 2017.

Références (2/2)

[11] Hugging Face. « SmolLM3-3B ». Fiche du modèle, 2025.

[12] B. Vignal. « Improving LLM Reliability in Healthcare Data Extraction ». Blog technique de Lifen, septembre 2025.

[13] P. Micikevicius *et al.* « FP8 Formats for Deep Learning ». arXiv:2209.05433, 2022.

[14] J. Lin *et al.* « AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration ». MLSys, 2024.

[15] E. Frantar *et al.* « GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers ». ICLR, 2023.

[16] W. Kwon *et al.* « Efficient Memory Management for Large Language Model Serving with PagedAttention ». SOSP, 2023.

[17] Y. Leviathan, M. Kalman, Y. Matias. « Fast Inference from Transformers via Speculative Decoding ». ICML, 2023.

[18] Y. Li *et al.* « EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty ». ICML, 2024.

[19] E. J. Hu *et al.* « LoRA: Low-Rank Adaptation of Large Language Models ». ICLR, 2022.

[20] TypeSafe AI. « Jev » et « System One models ». Documentation, docs.typesafe.ai, 2026.

[21] InfoQ. « TypeSafe AI Releases Jev: a Decision-Only Model That Returns Typed Probabilities Instead of Text ». Octobre 2026.